

Project Architecture Specifications

- Project Name: oga_glow (Flutter E-Commerce Mobile Application)
- Dart SDK: ^3.10.4 | Flutter Version: 3.10+ | Target Platforms: Android, iOS, Web
- State Management & Routing: GetX Pattern (Bindings, Controllers, Views)
- Network Layer: Dio HTTP Client with ApiClient Interceptor & Error Handling
- Storage: Flutter Secure Storage (Tokens/Secrets) & SharedPreferences (Theme/Settings)
- UI Framework: Material 3, Flutter ScreenUtil (Responsive Layouts), Google Fonts, Shimmer

1. Executive Architecture Overview

The oga_glow application is built following a Feature-Driven Clean Architecture combined with the GetX pattern in Flutter. This architectural model ensures loose coupling, high maintainability, testability, and a clear separation of concerns across all modules.

The codebase organizes infrastructure, shared utilities, data repositories, and UI screens into distinct top-level directories under lib/: app/, core/, data/, and features/. Each feature is encapsulated into its own self-contained folder.

Core Architectural Layers Breakdown

1. App Layer (lib/app/)

- App Entrypoint (lib/main.dart): Initializes Flutter engine bindings, storage singletons, and executes OgaGlowApp.
- Application Setup (lib/app/app.dart): Configures GetMaterialApp, initial routes, theme modes, and root bindings.
- Global Bindings (lib/app/bindings/initial_binding.dart): Lazily injects global singletons (ThemeController, ApiClient, Storage).
- Route Management (lib/app/routes/): Defines AppRoutes (route path strings) and AppPages (GetPage definitions with associated bindings).

2. Core Layer (lib/core/)

- Constants (lib/core/constants/): Static strings, FAQ models, policy constants, contact details, and order status text.
- Network Infrastructure (lib/core/network/): ApiClient (Dio configuration with auth interceptor), ApiEndpoints, ApiException, and ApiResponse wrapper.
- Storage Solutions (lib/core/storage/): SecureStorage for handling authentication tokens, session data, and encrypted local storage.
- Theme & Styling (lib/core/theme/): AppTheme (Light & Dark definitions), AppColors, AppTextStyles, and ThemeController for live theme switching.
- Shared Components & Utils (lib/core/widgets/ & lib/core/utills/): Reusable UI elements (BounceTap, PrimaryButton, CustomTextField, CustomAppBar, LoadingWidget) and utility helpers (DateFormatter).

3. Data Layer (lib/data/)

- Models (lib/data/models/): Data transfer objects and JSON serialization contracts (e.g. ProductModel).
- Providers (lib/data/providers/): Direct API HTTP service clients (ProductApiService, WishlistApiService).
- Repositories (lib/data/repositories/): Data abstraction layer bridging controllers and API providers (ProductRepository, WishlistRepository).

4. Feature Modules (lib/features/)

- Feature-Driven Structure: Modular directories dedicated to specific business domains (auth, cart, product, orders, etc.).
- Standardized Feature Pattern: Folder division into bindings/, controllers/, models/, repositories/, services/, views/, and widgets/.
- Reactive State Management: GetX GetxController managing reactive state (Rx variables) and providing data to Obx / GetView widgets.

2. Comprehensive Feature Modules Reference

The application consists of 14 distinct feature modules located inside `lib/features/`. Each feature follows a strict MVC & Binding pattern:

About Feature (`lib/features/about/`)

Contains brand information, company vision, and FAQ views. Components: `AboutScreen`, `AboutHeader`, `AboutBodyImageCard`, `AboutValuesSection`, and `AboutShimmerLoading`.

Address Feature (`lib/features/address/`)

Manages user delivery address book. Includes `AddressController`, `AddressModel`, `AddressRepository`, `AddressService`, and screens for viewing, creating, and selecting shipping addresses.

Authentication Feature (`lib/features/auth/`)

Handles user identity and session management. Modules: `Login`, `Registration`, `Forgot Password`, and `OTP Verification`. Driven by `AuthBinding`, `AuthController`, `AuthService`, `AuthRepository`, `UserModel`, and `AuthResponseModel`.

Cart Feature (`lib/features/cart/`)

Manages shopping cart items, quantity modifiers, subtotal computation, coupon discount application, and navigation to checkout. Managed by `CartController` and `CartScreen`.

Category Feature (`lib/features/category/`)

Facilitates product taxonomy discovery, category grid views, and category-filtered product listings via `CategoryController` and `CategoryScreen`.

Checkout Feature (`lib/features/checkout/`)

Executes order placement, delivery selection, price breakdown calculation, and payment processing through `CheckoutController`, `CheckoutService`, `CheckoutRepository`, and `CheckoutScreen`.

Home Feature (`lib/features/home/`)

The central app landing dashboard. Displays interactive banner carousel sliders, category quick-scroll rows, hot deals section, search bar, and featured product grids.

Legal Feature (`lib/features/legal/`)

Contains store policies, Terms of Service, and Privacy Policy views rendered using policy section cards (`PolicySection`, `PolicyBullet`, `PolicyHeader`).

Main Navigation Feature (`lib/features/main_navigation/`)

Root shell application scaffold. `MainNavigationController` controls tab switching across Home, Category, Cart, Wishlist, and Profile tabs without reloading views.

Orders Feature (`lib/features/orders/`)

Complete order management system. Features `OrderHistoryScreen`, `OrderDetailsScreen`, `OrderService`, `OrderController`, `OrderStatusChip`, and real-time tracking timeline widgets (`OrderTimeline`, `TrackingStep`).

Product Feature (`lib/features/product/`)

Full product details experience. Components: `ProductScreen`, `ProductImageSlider`, `ProductInfoSection`, `QuantitySelector`, `ProductActionButtons`, and product review management (`ReviewModel`, `ReviewRepository`, `ReviewService`, `ProductReviewsController`, `ReviewFormCard`).

Profile Feature (`lib/features/profile/`)

User account dashboard. Manages user info display (`UserInfoCard`), quick order links, and theme switching preference (`ProfileScreenThemeOption`).

Splash Feature (`lib/features/splash/`)

App startup sequence. `SplashScreen` and `SplashController` handle splash animations, verify persistent tokens, and direct the user to authentication or main navigation.

Wishlist Feature (lib/features/wishlist/)

Favorites management. WishlistController handles reactive bookmark toggling, empty state UI (EmptyWishlist), WishlistApiService, WishlistRepository, and saved items grid.

3. Data Flow & State Management Architecture

The application strictly enforces a reactive unidirectional data flow model powered by GetX:

Unidirectional Data Flow Pathway

1. User Interaction: User triggers an event in a View (e.g. clicks "Add to Cart" or "Place Order").
2. Controller Invocation: View delegates the action to its GetxController (e.g. cartController.addToCart()).
3. Service & Repository Layer: Controller calls the corresponding Repository / ApiService for data manipulation.
4. ApiClient Network Request: ApiClient (Dio) handles authorization headers, JSON payload formatting, and HTTP execution.
5. State Mutation: Controller receives parsed model objects and updates its reactive state variables (RxList, RxBool, RxInt).
6. Reactive UI Re-render: GetX Obx / GetBuilder widgets listening to the reactive variables update the UI instantly.

4. Developer Onboarding & Guidelines

Guidelines for Future Developers

- Creating New Features: Add a folder under lib/features/<feature_name>/ containing bindings/, controllers/, views/, and widgets/.
- Registering Routes: Add route string in AppRoutes (lib/app/routes/app_routes.dart) and map GetPage with binding in AppPages (lib/app/routes/app_pages.dart).
- Adding API Endpoints: Add endpoint path string in ApiEndpoints (lib/core/network/api_endpoints.dart) and implement call in dedicated ApiService.
- Dependency Injection: Avoid creating controllers directly inside views. Use Get.find<Controller>() or extend GetView<Controller>.
- Responsive Styling: Apply ScreenUtil extensions (.w, .h, .sp, .r) to all dimensions and font sizes for consistent cross-device layouts.
- Color & Typography Consistency: Utilize AppTheme, AppColors, and AppTextStyles. Avoid hardcoded Color(0xFF...) values.

5. Complete Project Folder & File Tree Map

The complete directory tree of the oga_glow repository is listed below:

```
[ROOT] oga_glow/
|-- [DIR] .github/
|   |-- [DIR] workflows/
|   |-- [FILE] build_apk.yml
|-- [DIR] android/
|   |-- [DIR] app/
|   |   |-- [DIR] src/
|   |   |   |-- [DIR] debug/
|   |   |   |   |-- [FILE] AndroidManifest.xml
|   |   |   |-- [DIR] main/
|   |   |   |   |-- [DIR] java/
|   |   |   |   |   |-- [DIR] io/
|   |   |   |   |   |   |-- [DIR] flutter/
|   |   |   |   |   |   |   |-- [DIR] plugins/
|   |   |   |   |   |   |   |-- [FILE] GeneratedPluginRegistrant.java
|   |   |   |   |-- [DIR] kotlin/
|   |   |   |   |   |-- [DIR] com/
|   |   |   |   |   |   |-- [DIR] example/
|   |   |   |   |   |   |   |-- [DIR] oga_glow/
|   |   |   |   |   |   |   |-- [FILE] MainActivity.kt
|   |   |   |-- [DIR] res/
|   |   |   |   |-- [DIR] drawable/
|   |   |   |   |   |-- [FILE] background.png
|   |   |   |   |   |-- [FILE] launch_background.xml
|   |   |   |-- [DIR] drawable-v21/
|   |   |   |   |-- [FILE] background.png
|   |   |   |   |-- [FILE] launch_background.xml
|   |   |-- [DIR] mipmap-hdpi/
```

```
| | | | | \-- [FILE] ic_launcher.png
| | | | | -- [DIR] mipmap-mdpi/
| | | | | \-- [FILE] ic_launcher.png
| | | | | -- [DIR] mipmap-xhdpi/
| | | | | \-- [FILE] ic_launcher.png
| | | | | -- [DIR] mipmap-xxhdpi/
| | | | | \-- [FILE] ic_launcher.png
| | | | | -- [DIR] mipmap-xxxhdpi/
| | | | | \-- [FILE] ic_launcher.png
| | | | | -- [DIR] values/
| | | | | \-- [FILE] styles.xml
| | | | | -- [DIR] values-night/
| | | | | \-- [FILE] styles.xml
| | | | | -- [DIR] values-night-v31/
| | | | | \-- [FILE] styles.xml
| | | | | \-- [DIR] values-v31/
| | | | | \-- [FILE] styles.xml
| | | | | \-- [FILE] AndroidManifest.xml
| | | | | \-- [DIR] profile/
| | | | | \-- [FILE] AndroidManifest.xml
| | | | | \-- [FILE] build.gradle.kts
-- [DIR] gradle/
| | | | | \-- [DIR] wrapper/
| | | | | \-- [FILE] gradle-wrapper.properties
-- [FILE] .gitignore
-- [FILE] build.gradle.kts
-- [FILE] gradle.properties
-- [FILE] local.properties
\-- [FILE] settings.gradle.kts
-- [DIR] assets/
| | | | | -- [DIR] animations/
| | | | | \-- [FILE] .gitkeep
-- [DIR] icons/
| | | | | \-- [FILE] .gitkeep
-- [DIR] images/
| | | | | -- [FILE] .gitkeep
| | | | | -- [FILE] auth_pattern.png
| | | | | -- [FILE] banner1.jpeg
| | | | | -- [FILE] banner2.jpeg
| | | | | -- [FILE] banner3.jpeg
| | | | | -- [FILE] brand_emblem.png
| | | | | -- [FILE] logo.png
| | | | | \-- [FILE] product_placeholder.png
\-- [DIR] svg/
| | | | | \-- [FILE] .gitkeep
-- [DIR] lib/
| | | | | -- [DIR] app/
| | | | | | | | | -- [DIR] bindings/
| | | | | | | | | \-- [FILE] initial_binding.dart
| | | | | | | | | -- [DIR] routes/
| | | | | | | | | | | -- [FILE] app_pages.dart
| | | | | | | | | | | \-- [FILE] app_routes.dart
| | | | | | | | | \-- [FILE] app.dart
-- [DIR] core/
| | | | | -- [DIR] constants/
| | | | | | | | | -- [FILE] about_constants.dart
| | | | | | | | | -- [FILE] contact_constants.dart
| | | | | | | | | -- [FILE] faq_constants.dart
| | | | | | | | | -- [FILE] faq_models.dart
| | | | | | | | | -- [FILE] legal_constants.dart
| | | | | | | | | -- [FILE] order_constants.dart
| | | | | | | | | \-- [FILE] order_models.dart
-- [DIR] extensions/
| | | | | \-- [FILE] theme_extensions.dart
-- [DIR] network/
| | | | | -- [FILE] api_client.dart
| | | | | -- [FILE] api_endpoints.dart
| | | | | -- [FILE] api_exception.dart
| | | | | \-- [FILE] api_response.dart
-- [DIR] storage/
| | | | | \-- [FILE] secure_storage.dart
-- [DIR] theme/
| | | | | -- [FILE] app_colors.dart
| | | | | -- [FILE] app_text_styles.dart
| | | | | -- [FILE] app_theme.dart
| | | | | -- [FILE] controllers
| | | | | \-- [FILE] theme_controller.dart
-- [DIR] utils/
| | | | | \-- [FILE] date_formatter.dart
\-- [DIR] widgets/
| | | | | -- [FILE] bounce_tap.dart
| | | | | -- [FILE] custom_app_bar.dart
| | | | | -- [FILE] custom_empty_state.dart
| | | | | -- [FILE] custom_network_image.dart
| | | | | -- [FILE] custom_snackbar.dart
| | | | | -- [FILE] custom_text_field.dart
| | | | | -- [FILE] fade_slide_transition.dart
```

Page 5 of 8

Page 6 of 8

Page 7 of 8

```
| | | | -- [DIR] controllers/
| | | | \-- [FILE] splash_controller.dart
| | | | \-- [DIR] views/
| | | | \-- [FILE] splash_screen.dart
| | | | \-- [DIR] wishlist/
| | | | |-- [DIR] bindings/
| | | | | \-- [FILE] wishlist_binding.dart
| | | | |-- [DIR] controllers/
| | | | | \-- [FILE] wishlist_controller.dart
| | | | |-- [DIR] views/
| | | | | \-- [FILE] wishlist_screen.dart
| | | | |-- [DIR] widgets/
| | | | | \-- [FILE] empty_wishlist.dart
| | | | | \-- [FILE] wishlist_item_card.dart
| | | | \-- [FILE] main.dart
|-- [DIR] test/
| | \-- [FILE] api_cache_and_stock_test.dart
| | \-- [FILE] auth_error_handling_test.dart
| | \-- [FILE] brand_features_model_test.dart
| | \-- [FILE] checkout_and_place_order_test.dart
| | \-- [FILE] live_api_suite_test.dart
| | \-- [FILE] review_and_contact_models_test.dart
| | \-- [FILE] widget_test.dart
|-- [DIR] web/
| | \-- [DIR] icons/
| | | \-- [FILE] Icon-192.png
| | | \-- [FILE] Icon-512.png
| | | \-- [FILE] Icon-maskable-192.png
| | | \-- [FILE] Icon-maskable-512.png
| | \-- [FILE] favicon.png
| | \-- [FILE] index.html
| | \-- [FILE] manifest.json
|-- [FILE] .flutter-plugins-dependencies
|-- [FILE] .gitignore
|-- [FILE] .metadata
|-- [FILE] analysis_options.yaml
|-- [FILE] API Documentation - OgaGlow & Wholcure (1).pdf
|-- [FILE] devtools_options.yaml
|-- [FILE] OGA_GLOW_Architecture_and_Folder_Structure.pdf
|-- [FILE] Place_Order_API-ogaglow_Documentation.pdf
|-- [FILE] pubspec.lock
|-- [FILE] pubspec.yaml
|-- [FILE] README.md
|-- [FILE] TODO.md
|-- [FILE] web_entrypoint.dart
\-- [FILE] Wishlist_API_Documentation.pdf
```